MAS: A durable communication protocol for auditable multi-agent workflows

Emmett Miller

Abstract

MAS is an open-source Rust implementation of a bounded communication protocol for multi-agent workflows. A versioned text envelope, durable mailbox and optional task profile separate message receipt from task completion. An assessment profile binds findings to exact candidates, evaluators, rubrics and evidence. The implementation provides identity-preserving retries, explicit routes, resource limits and runtime-observed expiry. We describe its representation, admission rules and integration boundary, and evaluate executable tests and finite Promela models with SPIN. The evidence supports specified communication and assessment controls within those bounds. It establishes neither exactly-once external effects nor improved agent reasoning. Source, models and reproduction scripts are available.

Keywords: multi-agent systems, communication protocol, durable messaging, model checking, auditable assessment

Code metadata

	Description	Value
C1	Current code version	Package 0.1.0; examined revision `a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1`
C2	Permanent link to code	https://github.com/emmett08/mas/tree/a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1
C3	Legal code licence	GPL-3.0-only; retained MIT notice for the original MAS Protocol Lab contribution
C4	Versioning system	Git
C5	Languages, tools and services	Rust, nom, SQLite; Promela/SPIN and a C compiler for verification; Python for optional host integration
C6	Compilation and environment	Pinned Rust 1.99.0; Cargo.lock; bundled SQLite; Linux for the verification harness. Optional PostgreSQL support is feature-gated.
C7	Developer documentation	https://github.com/emmett08/mas/blob/a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1/README.md
C8	Support email	Author confirmation required before submission; current public support: https://github.com/emmett08/mas/issues

1. Motivation and significance

An agent workflow needs to distinguish an accepted message, a completed task and a correct result. Conflating them makes a retry ambiguous: after losing a response, the caller cannot tell whether it should repeat delivery, repeat execution or reassess an existing output. MAS provides explicit records for these different questions. Its contribution is a reusable implementation combining bounded admission, durable identity retention, task correlation and content-bound assessments, with executable tests and finite-state models.

Agent2Agent (A2A) defines interoperable agent tasks, messages, artefacts and discovery; the Model Context Protocol (MCP) connects hosts to tools, resources and prompts [1, 2]. MAS addresses the local admission and retention of workflow exchanges. A2A extensions and application-specific MCP services can carry richer contracts; their existence does not establish the persistence behaviour of a particular deployment. MAS therefore complements interoperability interfaces rather than claiming a new universal agent

language.

The design follows Holzmann's separation of service, environment, vocabulary, encoding and exchange rules [3]. The service admits messages between registered participants and preserves their identity and delivery history. The environment contains a trusted host, protected credentials, functioning storage and participants whose outputs need not be correct. Agent-specific reasoning, tool permissions and substantive acceptance criteria remain application responsibilities. This boundary makes failures inspectable without treating a model's declaration of success as independent evidence.

2. Software description

2.1. Software architecture

The examined revision [4] implements the codec, SQLite repository, task transitions and capability gateway in Rust. The `nom` parser recognises the complete envelope before values enter the repository. The optional Python host invokes the Rust boundary; it does not reimplement mailbox semantics. Other host integrations and the optional PostgreSQL backend extend the deployment surface, but the measurements reported here concern the default-feature native implementation.

Figure 1 separates logical communication permission from physical transport. A directed graph determines which registered participants may exchange messages. Star, mesh and tree configurations use the same central repository. A mesh consequently permits peer-addressed messages without requiring peer-to-peer network connections. The gateway derives the run and actor from host-provisioned capabilities. Direct repository and administrative CLI calls instead trust the supplied actor, so database and administrative access must remain outside participant control.

Local interprocess communication (IPC) and the MAS envelope serve different purposes. The capability gateway uses JSON-line requests over standard input and output. The persistent trusted-host interface has a JSON header and length-delimited body. These interfaces transport operations on the repository; neither is the MAS message grammar or a remote authentication protocol.

2.2. Software functionalities

2.2.1. Representation and identity

Version 1 uses seven ordered fields: message identifier, run identifier, sender, recipient, version, kind and payload. The payload is a JSON object with an exact-integer subset of JSON's number syntax [5]. The parser rejects trailing input, duplicate decoded keys, malformed Unicode, fractions and

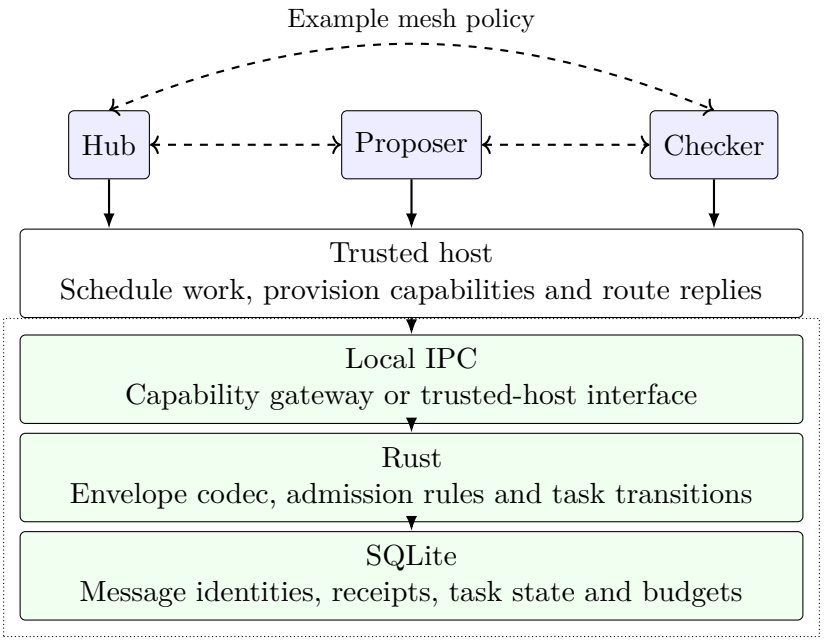


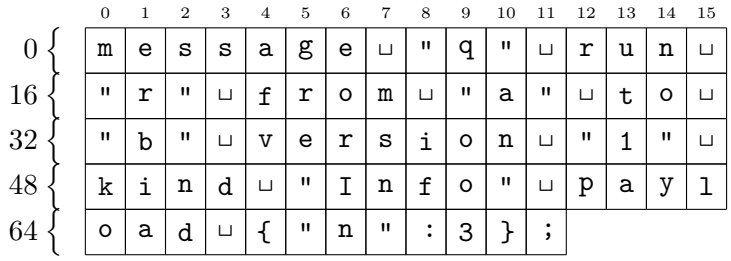
Figure 1: Logical routes and the local implementation boundary. Dashed links indicate permitted exchanges; solid arrows indicate requests to the central implementation. Host scheduling and external effects remain outside the mailbox transaction. Diagram source prepared with ChatGPT (OpenAI; model version unavailable) and rendered with LaTeX.

exponent notation. Limits are 65,536 UTF-8 bytes per envelope, 32 container levels and 1,024 decimal digits per integer. Applications can represent fractions as declared rational records or decimal strings.

Figure 2 shows a complete example, including delimiters and spaces. It uses `bytefield` to show octets of this particular text, not fixed-width binary fields. The canonical encoder normalises whitespace and object-key order. Message identity is the pair (r, i) of run and message identifiers. The repository retains the complete canonical envelope representation; equal content under an existing identity is an identical retry, whereas changed content is a conflict. Acknowledgement does not release that identity.

2.2.2. Admission, delivery and task state

A run freezes membership, routes, application profile, mailbox capacity, communication and call budgets, and a deadline. Algorithm 1 summarises SQLite admission. Identical retries retain their original acceptance even after the deadline or when the mailbox is full. New sends charge the byte budget using the stored JSON envelope representation; this accounting quantity differs from the text-wire length limit. A full mailbox refuses a new message for retry rather than discarding an accepted message.



76 octets; generic application profile; no trailing newline.

Figure 2: Exact UTF-8 bytes of a canonical generic-profile message. Each cell is one octet; row labels give decimal offsets. A visible space denotes byte 32. Empty final cells are outside the message, not padding. Field boundaries change when values change. Diagram source prepared with ChatGPT (OpenAI; model version unavailable) and rendered with LaTeX.

Algorithm 1 Admit a message into the SQLite repository

Input: Typed message m; host-bound actor a; repository D
Output: Original receipt, new receipt, or typed admission error

Reject an unknown run; reconcile its observed expiry
Encode and reparse m; derive canonical stored content c
Begin an immediate write transaction; load frozen run r
Require registered $a = m$.sender and recipient; require allowed route
if D contains $(m.\text{run}, m.\text{id})$ **then**
    Require retained content $= c$
    Commit; **return** original sequence with duplicate flag
end if
Reconcile the deadline again under the write transaction
if the run has expired **then**
    Commit expiry; **return** deadline rejection
end if
Require active version, remaining message/byte budgets and mailbox space
if the run uses the task profile **then**
    Require a valid transition from the retained task state
end if
Insert c; record the task transition when applicable; charge budgets
Commit; **return** new sequence with duplicate flag cleared

The pseudocode's requirements return errors on failure; an uncommitted write transaction rolls back. Independent expiry reconciliation and gateway attempt accounting can already have committed. Database failures yield an unknown admission outcome in the typed Rust API. A caller must reconcile the original identity: absence from one observation does not prove that an outstanding operation cannot commit. These semantics rely on SQLite's transaction and storage assumptions [6, 7].

Receive returns addressed, unacknowledged messages in insertion order and records receipt. Repeated receive can return the same message until acknowledgement. The optional `agent-task/1` profile additionally correlates each result or failure with one received request, reverses its participants and requires an open task. A Result reports either `completed` or `unresolved`; both close the task, and neither status independently verifies its output. Cancellation comes from the requester. Result, failure, cancellation and observed expiry compete for the first committed terminal transition (Figure 3). A reply requires receipt, not acknowledgement.

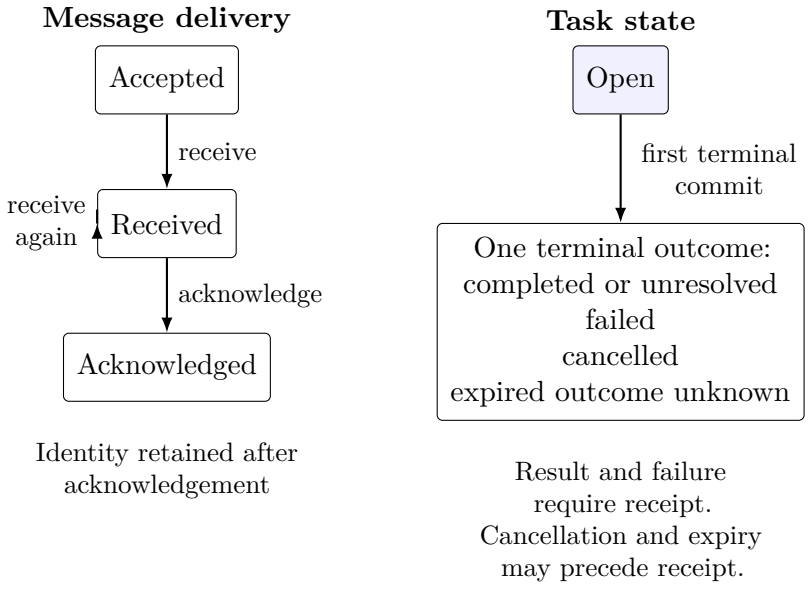


Figure 3: Two independent state records for a request. Receive enables a worker reply; acknowledgement releases mailbox capacity. Expiry closes an open task without inserting a terminal message. Closure does not establish whether an external effect occurred. Diagram source prepared with ChatGPT (OpenAI; model version unavailable) and rendered with LaTeX.

Expiry reconciliation closes open tasks as `expired_outcome_unknown`, even when message capacity is exhausted. It requires host activity and an available clock and database; it makes no real-time guarantee. A closed

request can remain in the mailbox as history with `actionable:false`. That value is a snapshot, not an exclusive execution lease. Payloads confer no additional execution authority. These distinctions limit the claim to identity-preserving admission and redelivery, without promising exactly-once external effects. This follows the end-to-end distinction between lower-level delivery and application-level completion [8].

2.2.3. Auditable assessments

The `assessment.v1` profile adds typed payload records without changing the version-1 grammar. Candidate, evidence, rubric and assessment records have explicit identities and schema versions. References bind both an identifier and a SHA-256 digest of the complete referenced record. Separate output and evidence-content digests bind the contained values. Canonicalisation preserves exact integers and array order, sorts object keys by Unicode scalar value and performs no Unicode normalisation. It is not RFC 8785 JSON Canonicalization Scheme conformance [9].

The contextual validator in Algorithm 2 checks authority and reference resolution after structural decoding. A completed assessment can contain failed criteria. An inconclusive finding has no score; an evaluator error has no findings. Scores are signed 64-bit integers within rubric bounds. The rubric and evaluator define their meaning; the protocol does not derive a pass threshold from a score range. The host must supply evaluator authority from its frozen invocation configuration, independently of the assessment payload.

An assessment travels inside the existing task Result's `output`. Its identity and evaluator must match the envelope's message identifier and sender. Revision validation requires a distinct candidate identity, an exact parent reference and precisely the supplied feedback references. A parent with no feedback permits a resampling control. The in-memory `AssessmentHistory` deduplicates equal records and rejects changed content under the same identity. Durable revision reservations, disclosure policy and execution belong to a host integration. The protocol records what assessment was supplied; it cannot establish what an evaluator knew or whether its judgement was correct.

3. Illustrative examples

3.1. A request, a restart and a result

The supplied star fixture registers a hub, proposer and checker. In a minimal sum exchange, the hub sends a request containing an operation and integer input; the worker receives it and returns a result correlated by `reply_to`. Restarting after receipt but before acknowledgement permits

Algorithm 2 Validate an assessment against retained context

Input: Assessment A; candidate C; rubric R; evidence set E;
host-authorised evaluator V

Output: Contextual validity or a specific validation error

Revalidate the structures of A, C, R, V
Require A.evaluator $= V$
Resolve A's candidate and rubric references against C, R
Require unique evidence identifiers; revalidate each record in E
if A.status $\neq$ error **then**
    Require one finding per rubric criterion
end if
for all findings f in A **do**
    Resolve criterion k in R; require k.measurement $= V$.kind
    Require each present score to lie within k's bounds
    for all evidence references e in f **do**
        Resolve e against E and compare its complete-record digest
    end for
end for
return valid

redelivery of the original request. Reusing its message identity with identical content adds no second record. Separate transition tests reject a different result attempting to close the same terminal task. The application checker must still compare the proposed sum with independently calculated arithmetic.

The repository's deterministic fixtures exercise real Rust parsing and SQLite exchanges. They do not execute live language models. This makes the delivery example reproducible without credentials while leaving model behaviour outside its claims. The recovery example reopens storage and observes deadline expiry. The capability-gateway example challenges sender spoofing and retains attempt limits across restart. Table 2 reports the observed outcomes; commands and exact output records accompany the paper.

3.2. Verification and reproducibility

The evaluation freezes revision `a61b3da` and combines implementation tests with selected executable abstractions. The reproduction script records source hashes, tool versions, commands and raw logs. Default-feature Rust tests exercise codecs, repository operations, gateway controls, observation and assessments. PostgreSQL, external providers and separate host repositories are outside this run.

SPIN checks assertions and temporal properties on finite Promela models [10]. Transport checks distinguish finite fault budgets from indefinitely recurring faults. Safety concerns routing, identity, capacity and receipt order. Progress means eventual acknowledgement of each accepted message under a finite fault budget. Task, transaction and expiry models cover conflicting terminal outcomes and recovery boundaries. Assessment checks use two candidate identities, two assessment identities, two content values, two evaluators, three delivery attempts and two revision reservations. The model abstracts committed host transitions and digest values. Its disclosure and revision properties impose obligations on host integrations, not guarantees supplied by this repository alone.

For each message identity i in the transport model, the progress requirement is

$$\mathsf{G}\Big(\text{accepted}_i \Rightarrow \mathsf{F}\,\text{acknowledged}_i\Big), \tag{1}$$

where G means always and F means eventually. Equation 1 concerns acknowledgement, not successful computation. The checked star has three participants, two message identities and two content values, with capacities one and two. The bounded model permits each message one lost acknowledgement and one restart; the unlimited-fault variant removes those bounds. Table 1 separates successful searches from expected counterexamples.

Table 1: Recorded verification outcomes. Counterexamples and witnesses are distinct from positive searches.

Evidence category	Result
Executed Rust tests	79 passed; 0 failed
Ignored Rust model-suite entrypoints	3
Offline executable fixtures	3 completed
Complete model searches with zero errors	17
Replayed assertion-defect counterexamples	19
Replayed liveness-defect counterexamples	2
Unlimited-fault progress counterexample	1
Replayed reachability witnesses	2

The 17 October 2026 run used Rust/Cargo 1.99.0, SPIN 6.5.2 and GCC 13.3.0. Each model command had a 30-second, 512-MiB limit and depth 100,000. Positive results concern complete full-state searches of finite abstractions. Defect counterexamples demonstrate detection of specified faults; the unlimited-fault counterexample refutes unconditional progress. Witnesses establish reachability. These results support the declared controls, with implementation tests supplying correspondence evidence. They establish neither

Table 2: Observed offline fixture outcomes. Participants are deterministic; no model provider is called.

Fixture	Recorded observation
Star	4 accepted messages; 748 charged bytes; 0 call reservations. All messages acknowledged; checker accepted the proposed sum.
Recovery	1 charged message; 1 task expired with unknown outcome. Delivery non-actionable; identical retry retained sequence 1.
Gateway	4 charged attempts, including a rejected sender forgery. Message received and acknowledged after restart; restart preserved the exhausted attempt budget.

unrestricted implementation refinement nor improved answer quality.

Negative controls introduce isolated defects into model copies; their intended assertions or acceptance cycles must be detected and replayed. Reachability witnesses establish that accepted assessments and successful revision paths are possible. Witness failures are deliberately constructed observations, distinct from successful safety searches. Searches that exceed a resource bound are inconclusive. No refinement proof links every Rust execution to the models, and model state counts are not deployment performance measurements.

4. Impact

MAS provides an experimental substrate for comparing agent workflows while retaining who sent each result, its task correlation and the assessment context. Researchers can vary logical routes without changing envelope semantics and can distinguish a delivery failure from a rejected substantive answer. Exact candidate and feedback references support comparisons between revision and budget-matched resampling. Such comparisons still require independent outcome measures, frozen evaluators and protected held-out cases; this paper reports no measured improvement in reasoning or incident diagnosis.

The implementation is appropriate when bounded work, durable retries and inspectable task histories justify a central authority. A direct function call is simpler for one synchronous trusted component. A conventional queue may suffice when task correlation and assessment records already exist elsewhere. A2A is relevant when independently hosted agents need a standard external interface; MCP is relevant when clients need standard tool access. The choice depends on the required boundary, not the number of agent roles.

Storage durability, trusted-host isolation and correct adapter behaviour remain premises. Cancellation cannot undo external actions, and expiry

preserves uncertainty about them. Large deployments, remote security, long-duration availability and comparative throughput require separate qualification. The present evidence establishes reproducible behaviour of the selected implementation and finite models, not community uptake or production capacity.

5. Conclusions

MAS separates communication permission, durable delivery, task closure and assessment validity. Its Rust implementation makes retries and terminal transitions explicit, while its assessment profile binds findings to exact retained context. Executable tests and model checking of finite abstractions support these controls within stated assumptions. The resulting records enable reproducible workflow experiments; evidence that feedback improves outcomes must come from those experiments.

Data and software availability

The source revision in the metadata table contains the implementation, tests and Promela models. The accompanying `paper/` directory contains the manuscript, vector figure sources, reproduction script and retained verification evidence. Synthetic fixtures require no personal data or model-service credentials.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

ChatGPT (OpenAI; the service did not expose a model version) assisted with source inspection, literature retrieval, manuscript drafting, explanatory diagram source, reproduction scripts and editorial checks. Diagrams are rendered from inspectable LaTeX source. Reported test outcomes derive from recorded commands. Final author review and responsibility declarations must be completed before journal submission.

References

[1] A2A Project, Agent2Agent (A2A) protocol specification, release v1.0.1, 28 May 2026; accessed 17 October 2026 (2026).
URL https://github.com/a2aproject/A2A/blob/v1.0.1/docs/specification.md

[2] Model Context Protocol contributors, Model Context Protocol: Specification, protocol revision 2025-11-25; accessed 17 October 2026 (2025).
URL https://modelcontextprotocol.io/specification/2025-11-25

[3] G. J. Holzmann, Design and Validation of Computer Protocols, Prentice Hall, Englewood Cliffs, NJ, 1991.
URL https://spinroot.com/gerard/popd.html

[4] E. Miller, MAS: AI agent communication in Rust, source revision a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1, 17 October 2026 (2026).
URL https://github.com/emmett08/mas/tree/a61b3da98ef3ef11996dd79b4a3a048ed6c98ce1

[5] T. Bray, The JavaScript Object Notation (JSON) data interchange format, RFC 8259, RFC Editor (Dec. 2017). doi:10.17487/RFC8259.
URL https://www.rfc-editor.org/rfc/rfc8259

[6] SQLite developers, Atomic commit in SQLite, accessed 17 October 2026.
URL https://www.sqlite.org/atomiccommit.html

[7] SQLite developers, Write-ahead logging, accessed 17 October 2026.
URL https://www.sqlite.org/wal.html

[8] J. H. Saltzer, D. P. Reed, D. D. Clark, End-to-end arguments in system design, ACM Transactions on Computer Systems 2 (4) (1984) 277–288.
URL https://web.mit.edu/Saltzer/www/publications/endtoend/endtoend.pdf

[9] A. Rundgren, B. Jordan, S. Erdtman, JSON canonicalization scheme (JCS), RFC 8785, RFC Editor, informational (Jun. 2020). doi:10.17487/RFC8785.
URL https://www.rfc-editor.org/rfc/rfc8785

[10] G. J. Holzmann, The model checker SPIN, IEEE Transactions on Software Engineering 23 (5) (1997) 279–295. doi:10.1109/32.588521.
URL https://spinroot.com/gerard/pdf/inprint/ieee97.pdf